

Linguaggio TNT

Manuale di Riferimento del Linguaggio

Elementi di Ingegneria dei Linguaggi di Programmazione 2025/2026

Domenico Iorio
Vittorio Landi
Francesco Moccaldi

Indice

1. Panoramica	3
2. Struttura di un file	3
3. Importazioni	3
3.1. Librerie di sistema	3
3.2. File locali	4
4. Tipi	4
4.1. Tipi primitivi	4
4.2. Tipi puntatore (<code>ref</code>)	4
4.3. Tipi array	4
4.4. Tipi struct (custom)	4
4.5. Il valore <code>null</code>	5
5. Struct	5
6. Funzioni	6
7. Istruzioni	6
7.1. Dichiarazione di variabile e costante	6
7.2. Istruzione <code>if</code> / <code>else</code>	7
7.3. Ciclo <code>while</code>	7
7.4. Ciclo <code>for</code>	7
7.5. <code>return</code>	8
7.6. <code>break</code> e <code>continue</code>	8
7.7. <code>defer</code>	8
7.8. <code>tnt</code> (panic)	9
8. Espressioni	9
8.1. Operatori e precedenza	9
8.2. Cast (<code>as</code>)	10
8.3. Operatore <code>addr</code>	10
8.4. Operatore di dereferenziazione (<code>-></code>)	10
8.5. Letterali array	11
8.6. Chiamate a funzioni C esterne (<code>c.</code>)	11
9. Letterali	12

1. Panoramica

TNT è un linguaggio di programmazione imperativo a tipizzazione statica che viene transpilato in C. La sintassi si ispira a linguaggi moderni come Rust, Swift e Zig, mantenendo un modello di esecuzione e un sistema di tipi compatibili con C. Il compilatore (transpiler) traduce il codice sorgente `.tnt` in C valido pronto per essere compilato con qualsiasi toolchain C standard.

Caratteristiche principali:

- Tipizzazione statica forte, senza conversioni implicite.
- Gestione esplicita dei puntatori tramite le keyword `ref`, `addr` e `->`.
- Accesso ai campi delle struct sempre tramite l'operatore `.` (il transpiler produce `->` quando necessario).
- Costrutto `defer` per la gestione garantita delle risorse.
- Costrutto `tnt` per la terminazione immediata del programma (panic).
- Interoperabilità con funzioni C esterne tramite il prefisso `c.`.

2. Struttura di un file

Un file sorgente TNT è composto da una sezione di importazioni (opzionale) seguita da un elenco di dichiarazioni di tipo e di funzione a livello globale. L'ordine è vincolato: le importazioni devono precedere qualsiasi altra dichiarazione.

```
<sezione importazioni>
<dichiarazioni struct e funzioni>
```

Il punto di ingresso del programma è la funzione `main`, che deve restituire `int`.

```
import stdio.h

fn main() -> int {
    printf("Hello, world!\n");
    return 0;
}
```

3. Importazioni

TNT distingue due tipi di inclusione: librerie di sistema e file locali. Tutte le importazioni devono comparire all'inizio del file, prima di qualsiasi altra dichiarazione.

3.1. Librerie di sistema

```
import <nome_libreria>
```

Viene tradotta in `#include <nome_libreria>`. Il nome può contenere lettere, cifre, underscore, slash e un'estensione opzionale.

```
import stdio.h
import stdlib.h
import string.h
```

3.2. File locali

```
import local <percorso_relativo>
```

Il percorso deve iniziare con `./` o `../`. Viene tradotta in `#include "percorso"`.

```
import local ./utils.h
import local ../common/types.h
```

4. Tipi

4.1. Tipi primitivi

TNT mette a disposizione i seguenti tipi base:

Tipo	Descrizione
<code>int</code>	Intero con segno
<code>float</code>	Numero in virgola mobile
<code>char</code>	Singolo carattere
<code>void</code>	Assenza di valore; utilizzabile solo come tipo di ritorno

4.2. Tipi puntatore (`ref`)

Un tipo puntatore si dichiara anteponendo `ref` a un tipo esistente. Corrisponde a un puntatore C (`T*`).

```
let p: ref int;           // puntatore a int
let s: ref char = "ciao"; // puntatore a char (stringa)
let q: ref Point;         // puntatore a struct Point
```

I tipi `ref` possono essere annidati: `ref ref int` corrisponde a `int**`.

4.3. Tipi array

Un array si dichiara con la sintassi `<tipo>[<dimensione>]`. La dimensione è opzionale se l'array viene inizializzato con un letterale.

```
let nums: int[5];           // array di 5 interi
let primes: int[5] = [2, 3, 5, 7, 11];
let chars: char[] = ['a', 'b', 'c']; // dimensione inferita
```

Un parametro di tipo `ref int[5]` denota un puntatore a un array di 5 interi (utile per passare array a funzioni per riferimento).

4.4. Tipi struct (custom)

I tipi definiti con `struct` (vedere Sezione 5) sono tipi validi in qualsiasi posizione dove è atteso un tipo.

4.5. Il valore `null`

Il letterale `null` è assegnabile a qualsiasi tipo `ref`. Corrisponde a `NULL` in C.

```
let ptr: ref int = null;

if (ptr == null) {
  tnt "puntatore nullo";
}
```

5. Struct

Una struct definisce un tipo composto con campi nominati. La dichiarazione avviene a livello globale.

```
struct <Nome> {
  <campo>: <tipo>;
  ...
};
```

I nomi delle struct e dei relativi campi devono essere unici all'interno del proprio scope.

```
struct Point {
  x: int;
  y: int;
};

struct Rect {
  origin: Point;
  width: int;
  height: int;
};
```

L'accesso ai campi avviene sempre tramite l'operatore `.`, sia su variabili dirette che su puntatori; il transpiler genera automaticamente `->` quando necessario.

```
fn translate(p: ref Point, dx: int, dy: int) -> void {
  p.x = p.x + dx;
  p.y = p.y + dy;
}

fn main() -> int {
  let pt: Point;
  pt.x = 0;
  pt.y = 0;
  translate(addr pt, 3, 4);
  return 0;
}
```

6. Funzioni

Una funzione si dichiara con la parola chiave `fn`, seguita dal nome, dalla lista di parametri tra parentesi, dal tipo di ritorno e dal corpo.

```
fn <nome>(<parametri>) -> <tipo_ritorno> {  
    <istruzioni>  
}
```

I parametri hanno la forma `<nome>: <tipo>` e sono separati da virgole. Per passare un array per riferimento si usa `ref <tipo>[<dim>]`.

```
fn add(a: int, b: int) -> int {  
    return a + b;  
}  
  
fn greet(name: ref char) -> void {  
    printf("Ciao, %s!\n", name);  
}  
  
fn scale(arr: ref int[4], factor: int) -> void {  
    let i: int = 0;  
    while (i < 4) {  
        ->arr[i] = ->arr[i] * factor;  
        i = i + 1;  
    }  
}
```

La funzione `main` deve essere sempre dichiarata e deve restituire `int`.

7. Istruzioni

7.1. Dichiarazione di variabile e costante

Le variabili si dichiarano con `let`, le costanti con `const`. Entrambe richiedono un tipo esplicito; l'inizializzatore è obbligatorio per `const` e opzionale per `let`.

```
let <nome>: <tipo> [= <espressione>];  
const <nome>: <tipo> = <espressione>;
```

```
let count: int = 0;  
let buffer: ref char = null;  
let coords: int[2] = [10, 20];  
const MAX_SIZE: int = 256;  
const PI: float = 3.14159;
```

Ogni variabile deve essere dichiarata prima del suo utilizzo. Non è consentita la ridichiarazione dello stesso nome nello stesso scope.

7.2. Istruzione `if` / `else`

```
if (<condizione>) {  
    <istruzioni>  
} else if (<condizione>) {  
    <istruzioni>  
} else {  
    <istruzioni>  
}
```

I rami `else if` ed `else` sono opzionali e concatenabili a piacere.

```
fn classify(n: int) -> void {  
    if (n < 0) {  
        printf("negativo\n");  
    } else if (n == 0) {  
        printf("zero\n");  
    } else {  
        printf("positivo\n");  
    }  
}
```

7.3. Ciclo `while`

```
while (<condizione>) {  
    <istruzioni>  
}
```

```
let i: int = 0;  
while (i < 10) {  
    printf("%d\n", i);  
    i = i + 1;  
}
```

7.4. Ciclo `for`

```
for (<init>; <condizione>; <aggiornamento>) {  
    <istruzioni>  
}
```

Tutti e tre i componenti sono opzionali. L'inizializzatore può essere una dichiarazione `let` (il cui scope è limitato al ciclo) o un'espressione.

```
for (let i: int = 0; i < 5; i = i + 1) {  
    printf("%d\n", i);  
}  
  
// Componenti vuoti  
let j: int = 0;  
for (; j < 5; j = j + 1) {
```

```
    printf("%d\n", j);
}
```

7.5. `return`

```
return [<espressione>];
```

Restituisce il controllo al chiamante con il valore specificato. Prima dell'effettiva restituzione, tutti i `defer` attivi nello scope corrente e negli scope annidati vengono eseguiti in ordine inverso (LIFO). In una funzione `void` il valore è omesso.

```
fn safe_divide(a: int, b: int) -> int {
    if (b == 0) {
        tnt "divisione per zero";
    }
    return a / b;
}
```

7.6. `break` e `continue`

`break` interrompe immediatamente il ciclo più interno. `continue` salta all'iterazione successiva. Entrambe sono valide solo all'interno di cicli `while` o `for`.

```
let i: int = 0;
while (i < 100) {
    if (i % 2 == 0) {
        i = i + 1;
        continue;
    }
    if (i > 9) {
        break;
    }
    printf("%d\n", i);
    i = i + 1;
}
```

7.7. `defer`

```
defer <istruzione_expr>;
defer { <istruzioni> }
```

Pianifica l'esecuzione di un'istruzione (o di un blocco) alla fine dello scope corrente, oppure prima di un `return`. I `defer` vengono eseguiti in ordine LIFO (l'ultimo dichiarato viene eseguito per primo). Il costrutto `tnt` non attiva l'esecuzione dei `defer`.

```
fn open_and_process() -> int {
    let fd: int = c.open("data.bin", 0);
    defer c.close(fd);           // eseguito all'uscita dalla funzione
}
```

```

let buf: ref int = c.malloc(128) as ref int;
defer c.free(buf);           // eseguito prima di close

// ... elaborazione ...
return 0;
}

```

In presenza di più `defer` in uno stesso scope, l'ordine di esecuzione è inverso rispetto all'ordine di dichiarazione.

```

fn main() -> int {
    defer printf("terzo\n");
    defer printf("secondo\n");
    defer printf("primo\n");
    return 0;
    // Output: primo, secondo, terzo
}

```

7.8. `tnt` (panic)

`tnt` termina immediatamente il programma. I `defer` attivi non vengono eseguiti. Accetta quattro forme di argomento:

Sintassi	Comportamento
<code>tnt "messaggio";</code>	Stampa il messaggio su stderr e termina con codice 1
<code>tnt <intero>;</code>	Termina con il codice di uscita specificato
<code>tnt <variabile>;</code>	Termina con il valore della variabile come codice di uscita
<code>tnt -><puntatore>;</code>	Stampa il valore puntato (intero) su stderr e termina con codice 1

```

fn main() -> int {
    let ptr: ref int = c.malloc(64) as ref int;

    if (ptr == null) {
        tnt "allocazione fallita"; // stampa su stderr ed esce con 1
    }

    let code: int = 2;
    tnt code;                // esce con codice 2
}

```

8. Espressioni

8.1. Operatori e precedenza

La tabella seguente elenca gli operatori in ordine di precedenza crescente (gli operatori in cima hanno precedenza minore).

Livello	Operatori	Descrizione
1	<code>=</code>	Assegnamento (associativo a destra)
2	<code> </code>	OR logico
3	<code>&&</code>	AND logico
4	<code>==</code> <code>!=</code>	Uguaglianza e disuguaglianza
5	<code><</code> <code>></code> <code><=</code> <code>>=</code>	Relazionali
6	<code>+</code> <code>-</code>	Addizione e sottrazione
7	<code>*</code> <code>/</code> <code>%</code>	Moltiplicazione, divisione, modulo (solo <code>int</code>)
8	<code>as</code>	Cast di tipo
9	<code>-</code> <code>!</code> <code>-></code> <code>addr</code>	Unari: negazione, NOT, dereferenziazione, indirizzo
10	<code>()</code> <code>.</code> <code>[]</code>	Postfissi: chiamata, accesso a campo, indicizzazione

L'assegnamento (`=`) è associativo a destra e può essere utilizzato come espressione.

8.2. Cast (`as`)

```
<espressione> as <tipo>
```

Converte esplicitamente un valore in un altro tipo. Non sono consentiti cast verso o da tipi `struct`.

```
let x: int = 3;
let f: float = x as float;           // int -> float
let i: int = 3.99 as int;            // float -> int (troncamento)
let p: ref int = c.malloc(40) as ref int; // void* -> ref int
```

8.3. Operatore `addr`

```
addr <espressione>
```

Restituisce l'indirizzo di una variabile o di un'espressione. Il tipo risultante è `ref T` dove `T` è il tipo dell'operando. Corrisponde all'operatore `&` del C.

```
let n: int = 42;
let ptr: ref int = addr n;

let pt: Point;
add_point(addr pt, addr pt);
```

8.4. Operatore di dereferenziazione (`->`)

```
-><espressione>
```

Accede al valore puntato da un tipo `ref`. Corrisponde all'operatore `*` del C. Per i puntatori ad array, `->arr[i]` accede all'elemento `i` dell'array puntato.

```
let n: int = 10;
let ptr: ref int = addr n;
let val: int = ->ptr;      // val == 10

->ptr = 20;                // modifica n tramite puntatore
```

```
fn double_all(arr: ref int[3]) -> void {
  let i: int = 0;
  while (i < 3) {
    ->arr[i] = ->arr[i] * 2;
    i = i + 1;
  }
}
```

8.5. Letterali array

```
[<expr>, <expr>, ...]
```

Crea un array con i valori specificati. Il letterale deve contenere almeno un elemento e tutti gli elementi devono avere lo stesso tipo primitivo. La dimensione dell'array viene inferita dal numero di elementi.

```
let primes: int[5] = [2, 3, 5, 7, 11];
let vowels: char[] = ['a', 'e', 'i', 'o', 'u'];
```

8.6. Chiamate a funzioni C esterne (`c.`)

Il prefisso `c.` consente di chiamare direttamente funzioni della libreria C standard o di qualsiasi altra libreria inclusa tramite `import`. Il controllo dei tipi viene disabilitato per tali chiamate. I tipi primitivi (`int`, `char`, `float`, `void`) possono essere passati come argomenti (utile per `sizeof`).

```
c.<nome_funzione>(<argomenti>)
```

```
let size: int = 10;
let buf: ref int = c.malloc(size * c.sizeof(int)) as ref int;
defer c.free(buf);

if (buf == null) {
  tnt "malloc fallita";
}
```

9. Letterali

Tipo	Esempi	Note
Intero	0, 42, 1024	Decimale non con segno; nessun prefisso ottale/esadecimale
Virgola mobile	3.14, 1.0, 2.5e-3	Notazione scientifica supportata
Carattere	'a', '\n', '\\'	Sequenze di escape: \n \t \r \\ \' \"
Stringa	"hello", "riga1\nriga2"	Tipo <code>ref char</code> ; stesse sequenze di escape
Booleano	true, false	Tipo <code>bool</code>
Null	null	Assegnabile a qualsiasi tipo <code>ref</code>

```
let c: char = '\n';
let s: ref char = "messaggio di errore";
let ok: int = 1;
let pi: float = 3.14159;
```